

A pipeline library for coding RL environments

What each method creates, how it works,
which upstream files to reuse, and where
to implement it in Repo2RLEnv.

Generation methods	One shared foundation
TMax, Endless Terminals, SETA, SWE-smith, SWE-Flow, CLI-Gym and TerminalWorld	LangGraph + Pi / OpenCode Remote Modal / Daytona execution Harbor tasks + independent validation

Includes new pipeline designs, upgrades to the six registered pipelines, annotated stage diagrams, exact source maps, quality gates and a staged implementation plan.

Prepared for Repo2RLEnv maintainers. Source and local code inspected on 10 September 2026. No upstream generators, container images or paid rollouts were executed for this report.

Guide to the report

Read 01-05 for the architecture; 06-16 for new recipes; 17-19 for existing pipeline upgrades; 20-26 for specialist extensions, validation, source pins and delivery milestones.

01 What we should build	3
02 The pipeline portfolio	4
03 One architecture for every recipe	5
04 The common task and evidence contract	6
05 Remote bootstrap and reusable images	7
06 TMax: compose coding tasks from skills	8
07 Endless Terminals: a simpler generation baseline	9
08 SETA Seed2Synth: turn source material into tasks	10
09 SETA Evol: expand an accepted task	11
10 SWE-smith: inject realistic repository defects	12
11 SWE-Flow: reconstruct a tested capability	13
12 Environment setup and repair	14
13 TerminalWorld: reconstruct a task from a trace	15
14 Test synthesis: the learner writes the tests	16
15 TaskPilot-inspired repository feature synthesis	17
16 Issue-to-environment as an input adapter	18
17 Improve the existing PR and commit pipelines	19
18 Improve the existing function pipelines	20
19 Strengthen security patch environments	21
20 Optional specialist pipelines and profiles	22
21 The shared verifier and acceptance system	23
22 Rollout calibration and honest yield	24
23 Where the implementation belongs	25
24 Implementation and validation plan	26
25 Upstream source register	27
26 Paper-method recipes and remaining boundaries	28

01 | What we should build

Build one Tasksmith execution and quality system, with generation recipes that create different kinds of coding tasks. A paper or repository becomes a recipe when it changes how a task is conceived or constructed. Bootstrap, verification, rollout review and publishing should be shared services.

The first additions should be `terminal_synth` with **TMax and Endless** recipes, `seed_synth` with **SETA**, and `task_evolve` with **SETA**. Together they cover creating tasks from skills, turning source material into tasks, and expanding an accepted task into a curriculum. Our existing PR/commit workflows remain the route for faithful reconstruction of real changes.

Three source findings make an adapter necessary. TMax exports Harbor tasks without an oracle script and deliberately omits teacher correctness validation. SETA parses an author-written review as its synthesis verdict. Endless proves initial-state executability but needs our reference and verifier-discrimination checks. These are useful generation foundations; our controller must own acceptance. [TMax generation README](#), [SETA per-seed pipeline](#), [Endless builder](#).

The target remains one faithful environment per suitable PR. More mutations or evolved children cannot compensate for failing to convert an input PR. Report those products separately. For synthetic inputs, retain the original seed denominator and every failed attempt; emitting a directory is not acceptance.

All diagrams describe the proposed Repo2RLEnv workflow. The source notes identify where that workflow differs from upstream. The workspace root is `/Users/adithya/projects/Repo2Reward`. Short local module paths such as `pipelines/`, `synthesis/`, `tasksmith/` and `bootstrap/` are relative to `src/repo2rlevn/`. Paths beginning `docs/`, `configs/`, `tests/` or `references/` are relative to the workspace root; section 23 gives the full integration map.

Reading route

- Sections 02-05: portfolio, shared architecture, task contract and remote builds.
- Sections 06-16: individual new pipelines and recipes, with implementation maps.
- Sections 17-19: improvements to existing PR, function and security pipelines.
- Sections 20-22: specialist extensions, verifier quality and rollout calibration.
- Sections 23-26: folder layout, implementation sequence, source pins and research-only methods.

02 | The pipeline portfolio

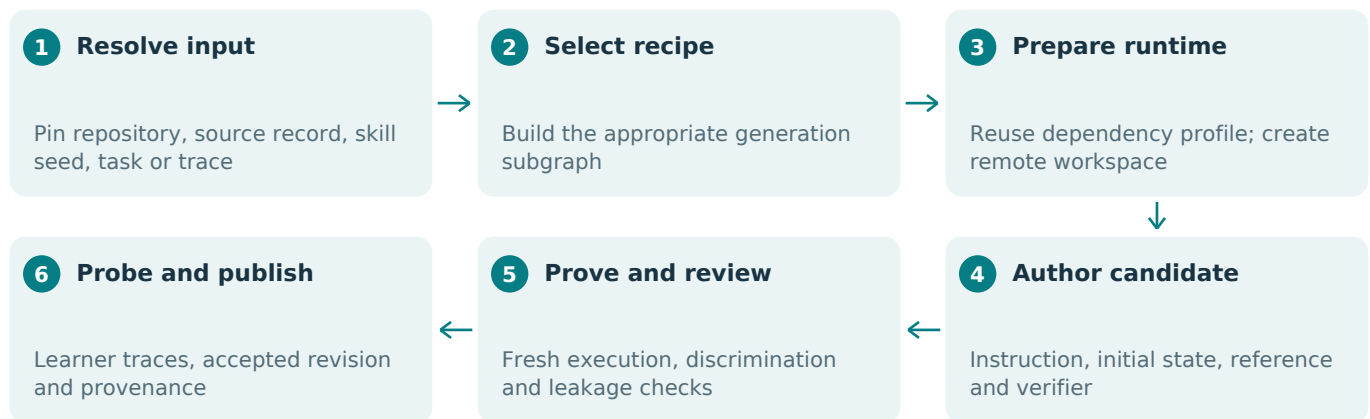
A family names the learner's task; a recipe names the generation method. This keeps compatible methods comparable and avoids maintaining a separate infrastructure stack for every paper.

Proposed family / recipe	Input and learner's job	Local status and priority
terminal_synth / tmax	Skills and domain -> complete a constructed coding/terminal task	New; first wave
terminal_synth / endless	Task distribution -> complete a staged terminal task	New; comparison baseline
seed_synth / seta	Notebook, Q&A or example -> implement a grounded task	New; first wave
task_evolve / seta	Accepted Harbor task -> solve a transformed child	New; first wave
repo_bugs / swesmith	Healthy repository -> diagnose and repair an injected defect	New; second wave
feature_rebuild / sweflow	Tested code slice -> reconstruct a coherent capability	New; second wave
env_setup / bare, repair	Repository or healthy image -> install or repair its environment	Existing draft; second wave
trace_to_task / terminalworld	Terminal recording -> reproduce its useful outcome	New; second wave
test_synthesis / mutation	Behavioral contract and hidden variants -> write discriminating tests	Existing draft; second wave
repo_feature / taskpilot-inspired	Repository -> implement a newly proposed feature	New method implementation; later
issue_runtime / resolved	Issue plus verified fix -> repair reported behavior	Existing draft; later adapter
problem_families / scaler-inspired	Executable generator -> solve sampled coding problems	Optional specialist extension

The six registered local pipelines are `pr_diff`, `pr_runtime`, `commit_runtime`, `code_instruct`, `equivalence_tests` and `cve_patches`. Tasksmith is a separate experimental PR graph. `env_setup`, `test_synthesis` and `issue_runtime` have draft RFCs but are not registered implementations. This distinction comes from the current local registry and RFC files, not documentation yield claims.

Shared services, not additional task families: remote bootstrap/cache, verifier authoring, adversarial hardening, calibration and publishing. Existing PR/commit and function families should absorb relevant SWE-gen, RepoLaunch, SWE-Next, R2E and SWE-Dev components.

03 | One architecture for every recipe



LangGraph owns durable stage transitions. Pi or OpenCode performs bounded coding work inside a stage. Harbor executes the resulting task. A harness can explore and edit freely within its assignment, but it returns artifacts and evidence references rather than declaring the task accepted. LangGraph's checkpointing supports durable execution; effects still need stable identities and recovery handling. [LangGraph durable execution](#).

The local `tasksmith/pipeline.py` already has discovery, construction, validation and repair nodes, backed by `state.py` and operation records. Its source and operation identities are tied to PRs. Generalize those identities and stage contracts before adding non-PR inputs. Preserve crash recovery, reservations and cleanup receipts while doing so.

The local `curation/external_agent.py` and `curation/runtimes/{pi,opencode}.mjs` already provide two author adapters. Reuse their tool bridge and tracing. Record **recipe, harness, model and prompt version separately** so a TMax-vs-Endless comparison does not accidentally compare different solver settings too.

Failure routing

- Dependency or build failure returns to the preparation node with logs and the failed command.
- Gold failure returns to reference/environment repair; baseline passing unexpectedly returns to scope or verifier repair.
- Unsupported assertions return to the public contract; leakage returns to packaging or instruction revision.
- A legitimate solver failure becomes difficulty evidence until the trace reveals a concrete environment defect.

Every revision invalidates evidence for changed artifacts. Repair only the affected stage, then rerun its dependent checks. An accepted parent is immutable; evolution creates another candidate.

04 | The common task and evidence contract

Generalize the local schema before wrapping external generators. `GenerationInput` currently requires `repo: RepoSpec`; `HarborTask` requires `oracle_diff`, writes a git-apply oracle, and accepts text-only auxiliary files. Those choices cannot represent all terminal, notebook, service or test-writing tasks cleanly.

Proposed contract	Required information	Local integration point
SourceSpec	Discriminated kind: repository, PR, issue, skill, source record, Harbor task or trace; immutable provenance	Extend <code>spec/input.py</code> ; add <code>spec/sources.py</code>
TaskBundle	Public instruction; initial files/image; reference script or patch; hidden verifier; binary assets and service manifest	Add <code>spec/task_bundle.py</code> ; normalize in <code>emitter/harbor.py</code>
BuildProfile	Dependency recipe, artifact hashes, resources, provider handles and compatibility evidence	Adapt <code>bootstrap/spec.py</code> , <code>tasksmith/providers.py</code>
ValidationRecord	Exact bundle hash, trial identity, test counts, results, failure class and logs	Generalize <code>tasksmith/contracts.py</code> , <code>validation.py</code>
LineageRecord	Source ID, parent ID, recipe, transformation and revision digest	Generalize <code>tasksmith/models.py</code> , <code>state.py</code>

Harbor supports a task directory with instructions, configuration, environment, tests and an optional solution. `Repo2RLEnv` should require an executable reference for this high-quality product even when the upstream format makes it optional. [Harbor task structure](#).

Migration rules

Keep existing repository configurations working through normalization. Do not use dummy repository URLs or empty patch oracles to satisfy old types. Add a whole-bundle digest covering instruction, solution, tests, fixtures, build recipe and resource settings; the existing emitter hashes only instruction and patch.

Converge `emitter/harbor.py` and `tasksmith/emit.py` on this bundle. Reuse the stricter `Tasksmith` artifact handling, but remove its PR-specific assumptions. Preserve generated author notes outside the learner bundle. Adapt `hub.py` and `curation/publish.py` to carry generic provenance and evidence.

First contract proof: round-trip a legacy PR task, a script-solution task and a task with binary fixtures. Parse every emitted task using the pinned Harbor version and show that changing a verifier or fixture changes its digest. Reject unsupported provider requirements before allocating a job.

05 | Remote bootstrap and reusable images



Separate **authoring bootstrap** from the learner's task. Cache dependencies independently of source revisions, then prove compatibility on each pinned revision. Repository-quarter or adjacent-commit grouping is a reuse heuristic; a matching lockfile alone does not prove compatibility with native build flags, drivers or system libraries.

Upstream code map: [RepoLaunch launch/core/workflow.py](#) for setup/verify/organize graphs; [launch/agent/setup/](#) for environment construction; [launch/agent/organize/](#) for rebuild commands and parsers; [adjacent-commit reuse](#). [SWE-Next src/swenext/repo_analysis/env_profile.py](#) and [generate_env_profiles.py](#) supply dependency-profile references. [SWE-rebench V2 prompts/installer/](#) supplies installer and parser prompts; it is a partial release.

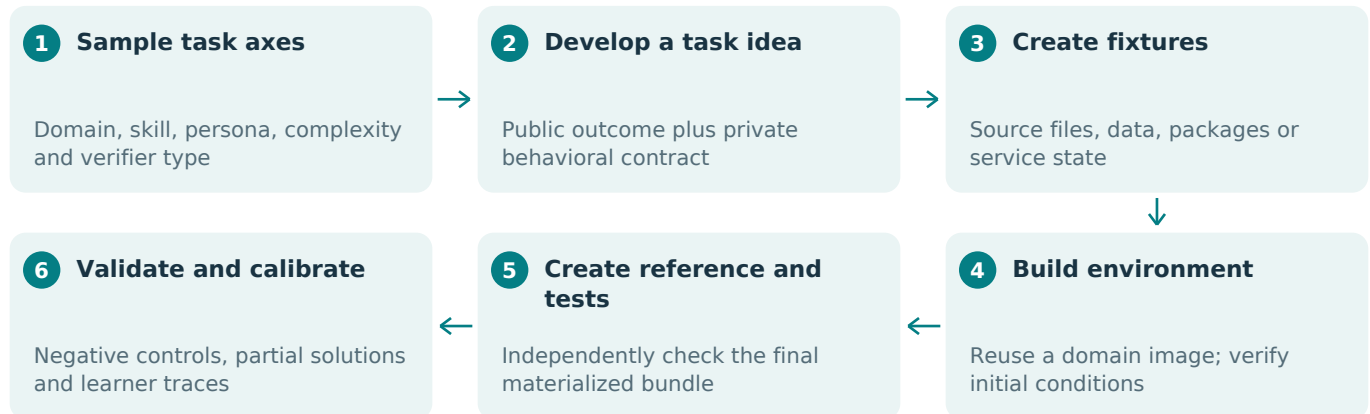
Provider choices: ordinary Dockerfile builds should use provider image APIs. For workloads that genuinely need a Docker daemon or nested services, Modal documents VM Sandboxes as its recommended Docker route, currently Beta. Daytona documents Dockerfile/image snapshots and Docker-in-Docker/Compose setup. Provider-level Compose capability does not imply Harbor's cloud adapter directly accepts a Compose task. [Modal VM guide](#), [Daytona snapshots and Docker guide](#).

Local work: retain `bootstrap/{runner,cache,presets,language}.py`; replace the legacy CLI's unconditional local Docker path with the remote provider interface. Extend `tasksmith/providers.py` and `conformance.py`. The inspected Modal adapter does not enable VM runtime, and its Daytona author profile fixes GPU count to zero. DinD, VM, GPU and service profiles require additional implementation and provider tests.

Milestone: build the same small repository on both providers; verify a clean restart and test collection; reuse its dependency image across two compatible revisions; invalidate on an incompatible dependency change. Confirm teardown and accounting. No images or fixtures are built on the local machine.

06 | TMax: compose coding tasks from skills

Add: `terminal_synth`, recipe `tmax`. **Learner job:** solve a realistic coding/terminal request in a constructed workspace. **Input:** domain, skills, fixture types and target difficulty. Initially constrain domains to programming, debugging, data processing and systems work relevant to coding.



Upstream implementation

`hamishivi/tmax`, `rl_data/`, Apache-2.0.

- `generator/task_template_gen.py`: compositional sampler and proposal prompts.
- `generator/fixture_gen.py`: fixture generation; move its host-side work into our remote sandbox.
- `generator/initial_state_test_gen.py` and `completion_test_gen.py`: separate initial and completion checks.
- `generator/apptainer_def_gen.py`: build/test feedback and retries; `containers/`: domain base recipes.
- `generate_tasks.py`: staged persistence; `scripts/analyze/convert_to_harbor.py`: known Apptainer sections, fixture staging and Harbor conversion.

Upstream primarily constructs Apptainer bundles. Its converter is useful reference code but does not emit `solution/solve.sh`. The method deliberately skips teacher correctness validation. We add a reference, prove it against the final exported artifact, and grade wrong/partial solutions. Prefer Docker-native authoring; a successful `.def` conversion is not a portability proof. [Converter](#), [generation policy](#).

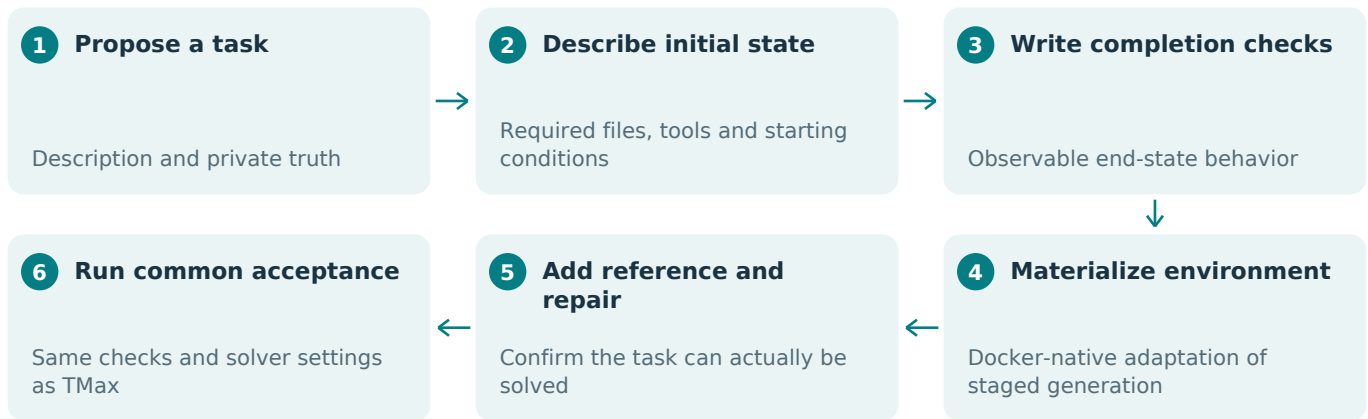
Proposed local files: `pipelines/terminal_synth.py`; `synthesis/terminal/tmax.py`; `synthesis/terminal/fixtures.py`; `synthesis/prompts/terminal/{idea,initial_state,completion}.md`; recipe config under `configs/pipelines/terminal_synth/`.

Example task: build a log-processing CLI that handles malformed records, preserves stable ordering and writes a documented summary format. Hidden checks exercise unseen logs and edge cases, not the exact reference code.

Milestone: five fixed skill seeds, each with a reference and fresh initial-state checks; trivial output hardcoding and omitted edge cases must lose reward. Compare accepted tasks, diversity and total cost with Endless on the same seed specification.

07 | Endless Terminals: a simpler generation baseline

Add: the `endless` recipe inside `terminal_synth`. This is a second construction strategy sharing the same output, remote execution and acceptance contract. It is valuable for measuring whether TMax's richer sampling produces enough benefit to justify its complexity.



Upstream implementation

[kanishkg/endless-terminals](#), Apache-2.0.

- Root `generate_tasks.py`: proposal, tests, environment and persistence stages.
- `generator/task_template_gen.py`, `initial_state_test_gen.py`, `completion_test_gen.py`: role prompts and generation functions.
- `generator/apptainer_def_gen.py`: active local build and initial-state testing. Despite its name, the inspected `iterate_def_template_batch` is single-shot.
- `generator/sample_solutions.py` and `generator/env.py`: solution sampling and execution.
- `generator/convert_to_harbor/convert_sif_docker.py`: LLM conversion, Docker build and initial-state checks; `add_reward_file.py`: reward-file adaptation.

The saved-SIF build in the root save stage is commented out, but earlier build/test validation remains active. The Docker converter's default candidate selection depends on prior positive o3 sampling results. Neither detail proves a trustworthy completion verifier. [Generation source](#), [conversion source](#).

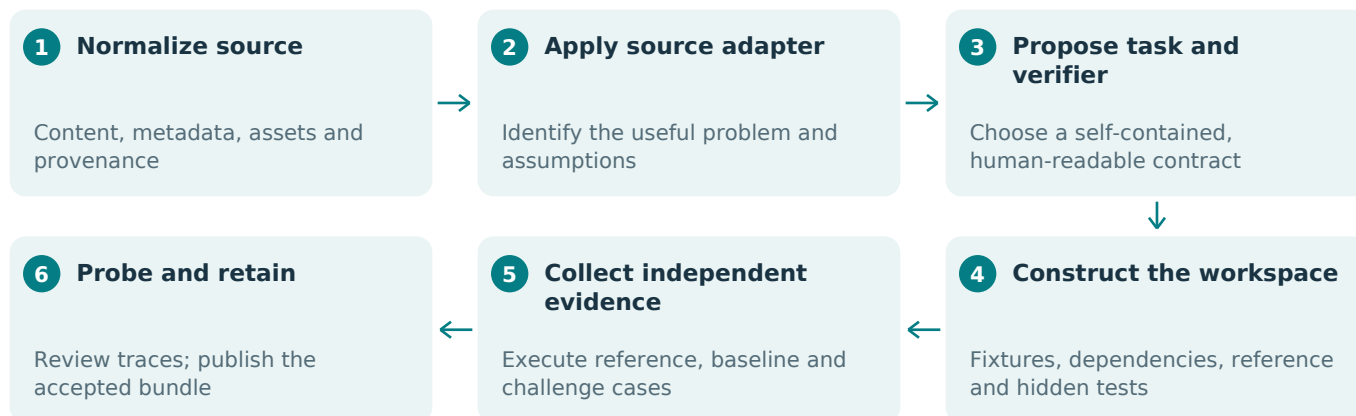
Proposed local files: reuse `pipelines/terminal_synth.py`; add `synthesis/terminal/endless.py` and a separate prompt/config recipe. Avoid another top-level `endless_pipeline` and a second cloud runner.

Example task: implement a configuration migration command that handles nested files, preserves unrelated fields and is idempotent. The output is a runnable script task with hidden fixture-based checks.

Milestone: run the same five skill seeds and resource envelopes as TMax. Record proposals, build success, valid references, accepted environments, solver behavior and per-stage cost. Use equivalent budgets and repeated samples before choosing a default; five seeds demonstrate the interface, not statistical superiority.

08 | SETA Seed2Synth: turn source material into tasks

Add: `seed_synth`, recipe `seta`. **Learner job:** implement a useful outcome grounded in a notebook, technical question or example. The source supplies context and constraints; its answer belongs in authoring evidence rather than learner-visible instructions.



Upstream implementation

[camel-ai/seta](#), [datasynth/seed2synth_pipeline/](#), Apache-2.0.

- `seed2task_pipeline.py`: idea-to-datapoint flow and review parsing.
- `agents/claude_agents.py`: role construction and source adapter map.
- `agents/seed2idea_prompts/`: Kaggle notebook, Stack Overflow, Unix/Linux SE and NL2Bash adapter prompts.
- `agents/datapoint_agent_guide/agent.md`: artifact-construction and cross-file consistency guidance.
- `seed2synth_orchestrator.py`, `seed2synth_config.py`, `run_orchestrator.py`: inventory, configuration and resumable producer/consumer organization.

Adaptation: the pipeline parses `judge_report.md` written by the same datapoint author. The guide asks the agent to run Harbor, but that is weaker than independently owned execution receipts. Its authoring path uses local Claude SDK tools; separate Daytona training integration is not evidence that generation already runs remotely. Extract prompts and adapters into Pi/OpenCode author stages and use our remote runtime. [Per-seed implementation](#).

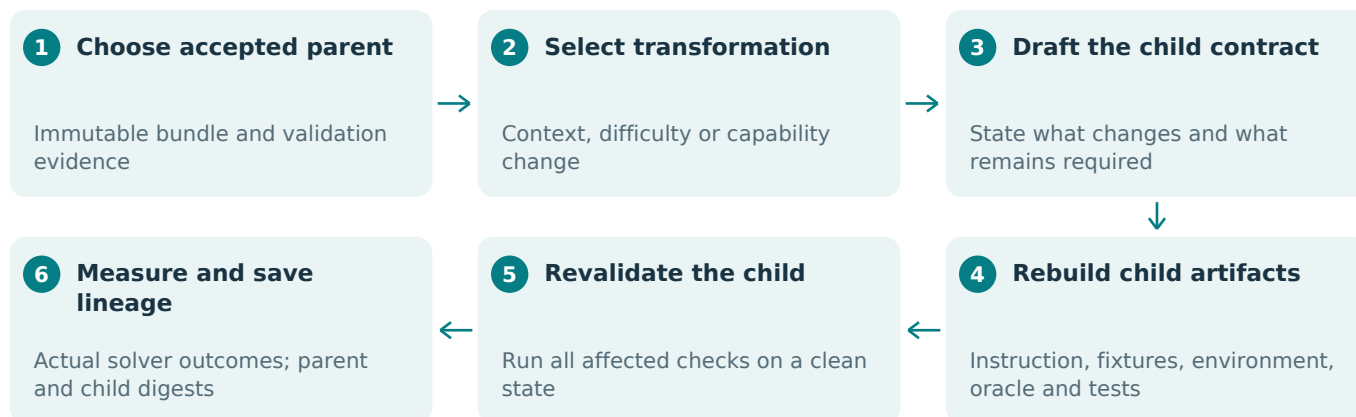
Proposed local files: `pipelines/seed_synth.py`; `synthesis/seeds/{normalize,seta}.py`; `synthesis/seeds/adapters/{notebook,qa,repo_docs,repo_examples}.py`; `synthesis/prompts/seeds/`. Repository-docs and examples adapters are our additions, not an existing SETA feature.

Example task: turn a notebook's fragile ingestion logic into a reusable streaming loader, with bundled sample data and explicit missing-value behavior. Hidden tests vary sizes, schemas and malformed input.

Milestone: one notebook and one Q&A task rebuild offline with all required assets. Neither the original answer nor notebook outputs may serve as leaked solutions. The reference passes, a plausible partial implementation fails, and tests accept a valid alternative implementation.

09 | SETA Evol: expand an accepted task

Add: task_evolve, recipe seta. **Learner job:** solve a new child task that changes a capability, constraint or context. Evolution is a dataset expansion product; the parent PR conversion stays intact.



Upstream implementation

[camel-ai/seta](#), [datasynth/evol_pipeline/](#), Apache-2.0.

- `evol_task_pipeline.py`: child task generation and datapoint construction.
- `agents/evol_agent_base_prompt.md`: transformation role.
- `agents/evol_strategy_prompts/`: increase/decrease difficulty, context changes, combined changes and slight changes.
- `agents/claude_agents.py`: author roles; `agents/traj_judge_prompt.md`: trace-review guidance.
- `evol_orchestrator.py`, `evol_config.py`, `run_evol_orchestrator.py`: orchestration and configuration.

The public workflow supports configured strategies and explicit output-to-input chaining. It does not establish a learned curriculum scheduler, and its author-report verdict needs the same replacement as Seed2Synth. A requested difficulty increase must be checked behaviorally; it is only a transformation label until measured.

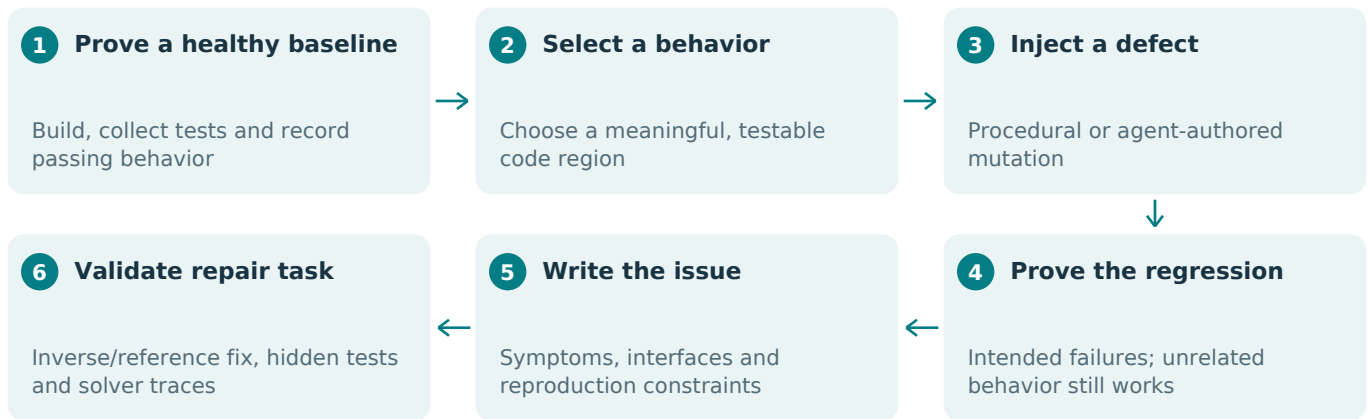
Proposed local files: `pipelines/task_evolve.py`; `synthesis/evolution/{seta,lineage,operators}.py`; `synthesis/prompts/evolution/`; optional `synthesis/evolution/dataarc.py`. The latter can reference [DataArc's sdgsystem/agentive_data/terminal_bench.py](#), on SynAgentiveData, for few/self/evol artifact generation. The Envs-FORGE paper's MILP selector was not found in that code.

Example task: evolve a single-file parser into a streaming parser with bounded memory, keeping its existing observable semantics. Add unseen large inputs and a declared measurement protocol.

Milestone: generate one context child and one capability child from the same accepted parent. Prove parent bytes are unchanged, each child reference passes, the parent's old solution fails at least the newly required behavior when applicable, and related lineages stay in the same data split.

10 | SWE-smith: inject realistic repository defects

Add: `repo_bugs`, recipe `swesmith`. **Learner job:** diagnose a meaningful bug and repair it. **Input:** a tested, healthy repository snapshot. The original implementation supplies a reference restoration; the injected defect creates the initial state.



Upstream implementation

[SWE-bench/SWE-smith](#), MIT.

- `swesmith/bug_gen/`: `procedural/generate.py`, `llm/modify.py`, `llm/rewrite.py` and `mirror/generate.py` for distinct mutation strategies.
- `swesmith/issue_gen/{generate,get_from_tests}.py`: convert defects and failures into issue descriptions.
- `swesmith/profiles/`: supported repository setup and execution profiles.
- `swesmith/harness/valid.py` and `grading.py`: execution and validity reporting.
- `scripts/bug_gen_modal.py`: a substantive Modal generation/validation implementation; use the actual filename, since its embedded run comment still names `bug_gen.py`.

Upstream has real generation infrastructure, including a remote route. It still assumes prepared repository profiles; it does not make arbitrary repository setup free. A mutation killed by existing tests proves a difference, not that the resulting task is realistic or that the verifier covers all valid fixes. [Modal source](#), [validity harness](#).

Proposed local files: `pipelines/repo_bugs.py`; `synthesis/bugs/{swesmith,operators,issue_writer}.py`; reuse the shared profile store and `tasksmith/validation.py`. Keep private inverse patches and mutation notes out of the learner snapshot.

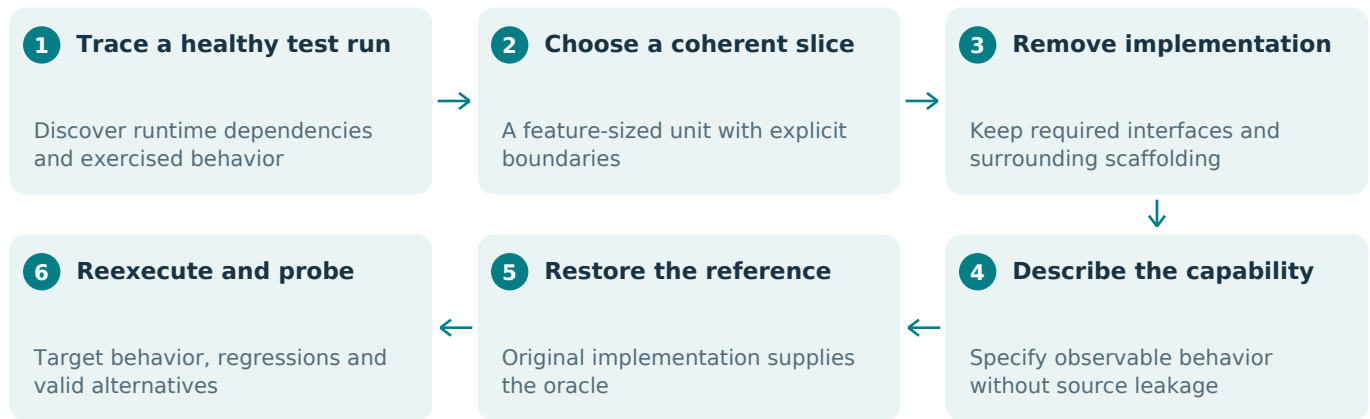
Example task: a connection pool mishandles cancellation and exhausts available connections. Verify resource recovery and concurrent behavior, rather than simply checking that the source matches the pre-mutation code.

Milestone: inject five behavioral defects into one cached repository. Each must produce an attributable failure and a successful reference restoration. Reject syntactic breakage, dead-code mutations and defects solved by deleting the requested functionality. Challenge graders with plausible partial fixes.

SWE-Synth is an additional mutation-design reference, but no root license was found in the prior audit. Prefer the clearly licensed SWE-smith implementation for direct code reuse; do not silently vendor ambiguous source.

11 | SWE-Flow: reconstruct a tested capability

Add: `feature_rebuild`, recipe `sweflow`. **Learner job:** implement a coherent capability that has been removed from a working repository. This extends beyond a single function while keeping a strong reference implementation and existing test evidence.



Upstream implementation

[Hambaobao/SWE-Flow](#), MIT, and [SWE-Flow-Trace](#), MIT. Exact source locations:

- `sweflow_trace/python/{trace,hooks}.py` in the Trace repository: `pytest` execution and call recording.
- `sweflow/extensions/python/{rdg,schedule}.py` in SWE-Flow: runtime dependency graph and development schedule.
- `sweflow/extensions/python/create_codebase.py` and `helper/codebase.py`: skeletons, partial repositories, patches and test groups.
- `sweflow/extensions/python/create_specification.py` and `data/specification/system-prompt.md`: behavioral specification authoring.

Runtime dependency graphs and schedules define the decomposition. Generated fail-to-pass/pass-to-pass labels are selection metadata until reexecuted on our actual initial and reference states. Async and multiprocessing traces need additional care; start with ordinary Python test execution. Keep independently generated cases to challenge solutions that pass only the traced examples.

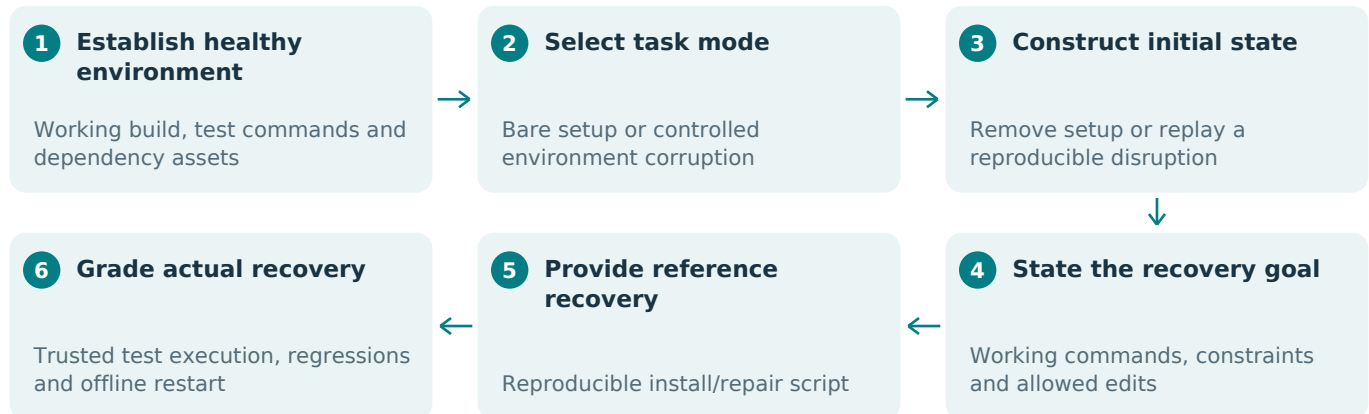
Proposed local files: `pipelines/feature_rebuild.py`; `synthesis/features/{trace_graph,slice,mask,specification}.py`; `reuse pipelines/_function_extractor.py` where applicable, without forcing function-only boundaries. Add prompt templates for observable requirements and scope review.

Example task: restore a serializer's nested object support, including type handling, cycle behavior and error reporting. Supply the public API and examples, while withholding the original implementation and hidden cases.

Milestone: one feature slice spanning multiple functions. The masked state must retain unrelated functionality, the original restoration must pass, and an implementation that handles only the happy path must fail. A trace that misses a dependency should cause a scope repair, not an instruction that asks the learner to guess missing behavior.

12 | Environment setup and repair

Implement the existing `env_setup` draft with two recipes. `bare` asks the learner to make a repository build and test successfully. `repair` starts from a corrupted dependency/configuration state, following CLI-Gym's environment inversion method. These teach different skills from fixing source code.



Upstream implementation

[LiberCoders/CLI-Gym](#), MIT. It builds faulty environments from healthy SWE-smith images and emits an older Terminal-Bench layout. Exact source locations:

- `src/cli_gym/cli.py`: runtime, test inventory, destruction and assembly call order.
- `src/cli_gym/build_destruction_task/{extract_uts,gen_destruction_prompt,gen_destruction_task}.py`: disruption construction; CLI-Gym/build_destruction_task/: prompt/task templates.
- `src/cli_gym/assemble_problem_instance/`: `destruction_result_parser.py` and `assemble_problem_instance.py` for results, instructions and broken-image assembly.
- `src/cli_gym/utils/docker_utils.py`: local Docker operations to replace with the remote interface.

The inspected assembly path grades selected failed tests and retains passing-test metadata separately; it does not produce our required reference restoration script. Some parser outcomes retain cases where pytest did not run. Our adapter must distinguish a legitimate environment-repair symptom from evidence that grading itself failed, and prove healthy -> broken -> restored in fresh instances.

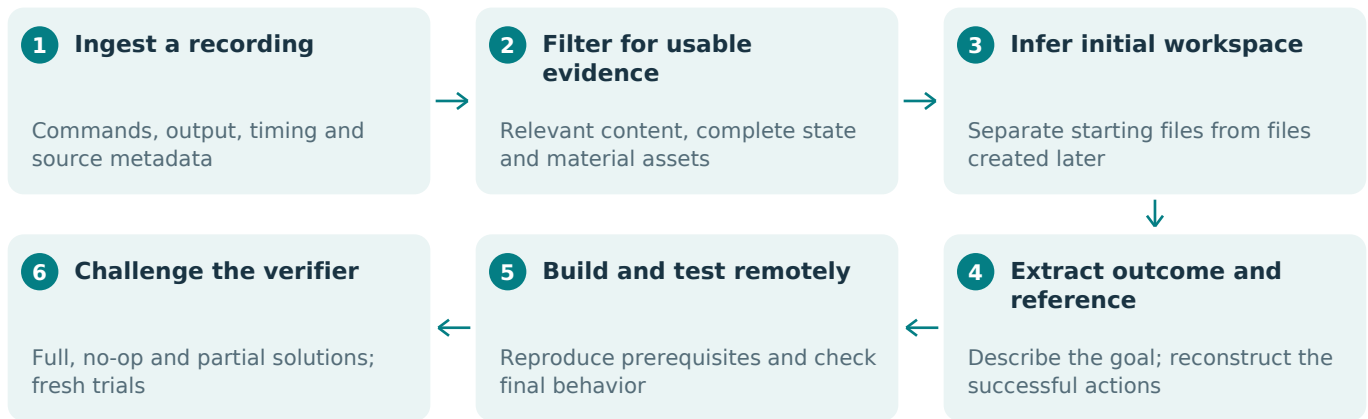
Proposed local files: `pipelines/env_setup.py`; `synthesis/environment/{bare,cli_gym,corruption,restore}.py`; update `docs/rfcs/0008-env-setup.md`; add setup/repair recipe configs. Authoring bootstrap still runs first so the producer knows a correct, reproducible environment exists.

Example task: repair package installation and plugin discovery so the documented CLI and selected suite run successfully using bundled dependencies. State whether changing source, lockfiles or system configuration is allowed.

Milestone: a deterministic corruption replay fails for the documented reason; the reference repair restores intended and regression checks. Grading reads trusted commands/results, not a learner-written success file. Offline package wheels or archives are sufficient, and a clean restart still works.

13 | TerminalWorld: reconstruct a task from a trace

Add: `trace_to_task`, `recipe terminalworld`. **Learner job:** achieve the useful outcome shown in a terminal recording, from a reconstructed initial workspace. Start with recordings that include enough evidence to reproduce the necessary state.



Upstream implementation

[EuniAI/TerminalWorld](#), Apache-2.0. This is the recording-derived project, distinct from the similarly named skill-derived Terminal-World paper.

- `data_retrieval/`: page scraping, downloads and recording parsers.
- `data_filtering/`: duration/value filters, external-URL and sensitive-content detection.
- `task_synthesis/generate_instruction.py`, `extract_solution.py`: outcome and reference construction.
- `environment_building/build_environment.py`, `analyze_recording.py`: environment reconstruction.
- `test_generation/generate_tests.py`, `refine_task.py`, `skill/`: verifier construction and task refinement.

The full/no-op/partial-solution design is particularly useful. For service or configuration tasks, avoid rewarding learner-written status dumps merely because they match a target snapshot; interrogate the real service or capture state through trusted tooling. Recording-derived prerequisites remain hypotheses until reproduced. [Generation and refinement source](#).

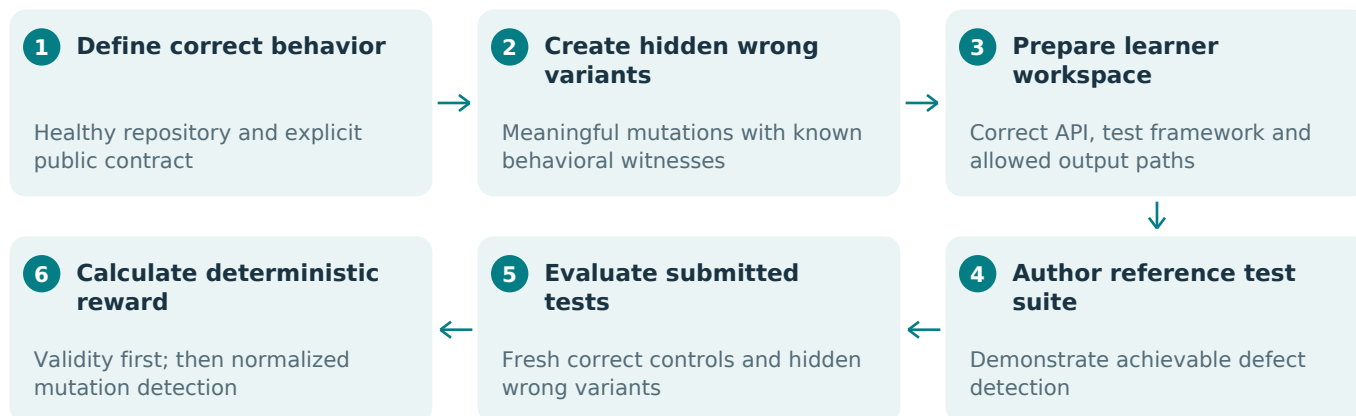
Proposed local files: `pipelines/trace_to_task.py`; `synthesis/traces/{ingest,reconstruct,terminalworld}.py`; adapters for asciinema initially. A future adapter for our coding-agent traces is an extension, not existing upstream support.

Example task: reconstruct a package migration demonstrated in a terminal session, then ask the learner to perform the migration against the original fixture project. Verify behavior and compatibility after migration.

Milestone: a recording-derived task whose initial files and dependencies can all be explained by evidence and reproduced offline. The reference succeeds, a partial replay fails an essential requirement, and grading does not depend on terminal timing or exact command strings.

14 | Test synthesis: the learner writes the tests

Implement: `test_synthesis`, initially recipe mutation. **Learner job:** write a test suite that passes correct implementations and catches hidden behavioral defects. This is distinct from authoring grading tests for other pipelines.



Upstream and local references

[SWE-Mutation framework/mutation.py](#), [agents/configs/mutation.yaml](#) and [agents/configs/test_generation.yaml](#) offer mutation/challenge construction; MIT. [R2E src/r2e/generators/testgen/genexec.py](#) offers generation/execution feedback; MIT. These are components for our task type, not a turnkey implementation of the local RFC.

Proposed local files: `pipelines/test_synthesis.py`; `synthesis/testing/{variants,oracle_suite,grader}.py`; update `docs/rfcs/0009-test-synthesis.md`. Keep correct and mutated variants in the trusted grading environment. Transfer only the learner's allowed test files.

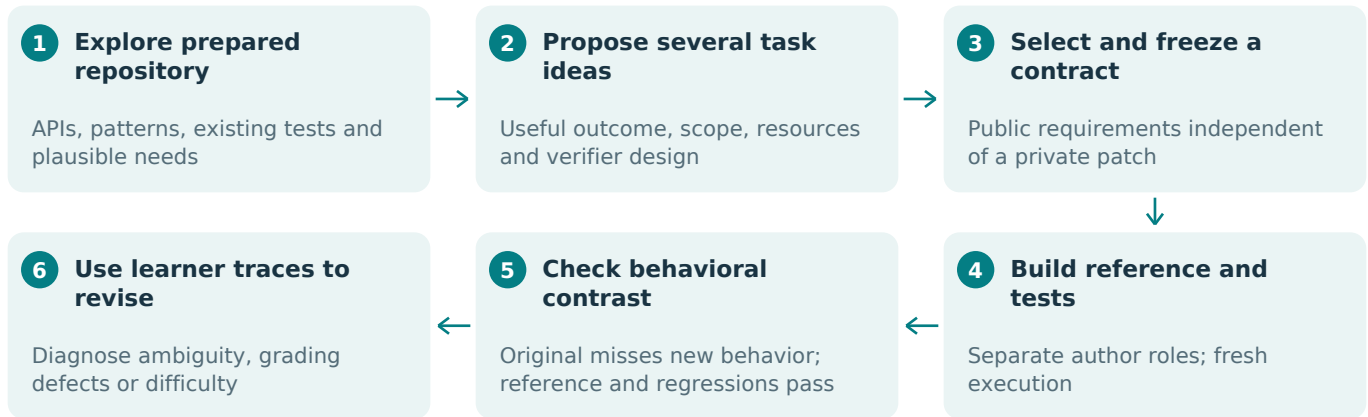
The current draft needs correction before implementation: a source-changing commit is not automatically a bug/fix pair; a refactor can preserve behavior. Its additive coverage term can reward tests that do not discriminate. Replace that with a hard validity gate followed by a normalized score over non-equivalent hidden variants. Coverage is diagnostic or an optional secondary metric after validity, not a substitute for catching errors.

Proposed reward: zero if tests do not collect, fail a correct control, violate submission restrictions or time out. Otherwise reward the weighted fraction of hidden wrong variants detected by attributable behavioral assertions. A bounded exception check may be legitimate; generic import failure must not count as detecting every mutation.

Milestone: the grader assigns zero to empty tests, `assert False`, tests that pass everything, and variant-fingerprinting shortcuts. A valid suite passes several correct implementations and detects meaningful unseen mutations. Deduplicate equivalent mutants before scoring.

15 | TaskPilot-inspired repository feature synthesis

Add later: `repo_feature`, recipe `taskpilot-inspired`. **Learner job:** implement a plausible new repository feature or behavioral improvement. It does not require an existing PR, but it requires a carefully specified new contract and a newly authored reference solution.



TaskPilot describes repository task creation with reference patches, hidden fail-to-pass tests, pass-to-pass checks and learner feedback. Its generator model identities are not fully disclosed, and no official generator implementation was located in this review. This would be our implementation of the paper method, not a wrapper around released TaskPilot code. [TaskPilot / FrogNano paper](#).

Reusable implementation components

- [SWE-gen `src/swegen/create/claude\_code\_runner.py`](#): bounded task construction and Harbor validation prompts, adapted from a PR setting.
- [THUDM SWE-Dev `swedev/testcases/`](#): behavioral descriptions, tests and revision after execution.
- Local Tasksmith discovery, authoring and validation components, generalized to repository identities.

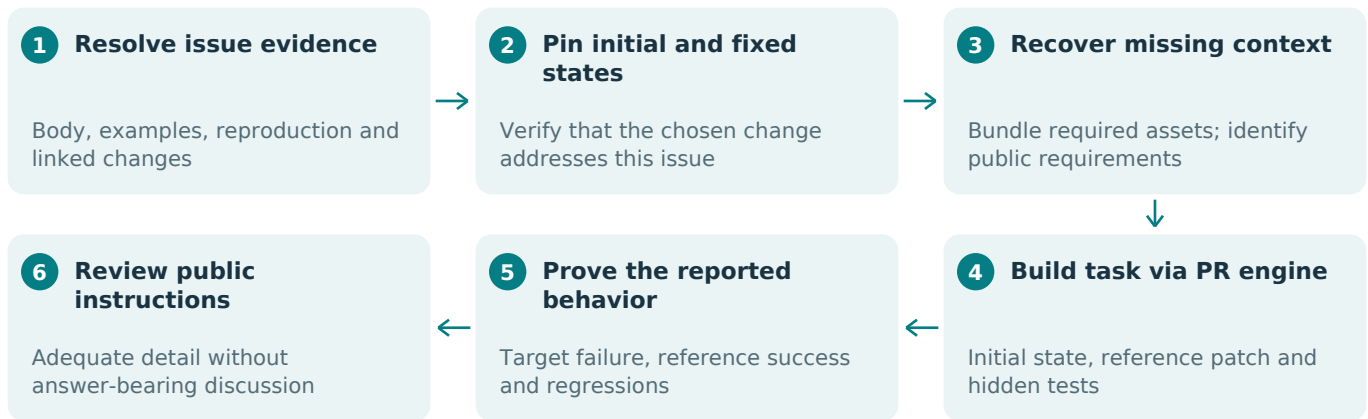
Proposed local files: `pipelines/repo_feature.py`; `synthesis/features/{proposal,contract,reference}.py`; `synthesis/prompts/features/`; integrate the shared trace-calibration stage. Keep tests and reference writers on the same public contract while withholding each other's private artifacts during first drafts.

Example task: add a batch operation to an existing client, defining ordering, errors and partial failures. Verify with a local service fixture and independent boundary cases.

Milestone: one novel feature with a precise public contract, successful reference and meaningful baseline failure. A second implementation written from the public task alone should be gradable fairly. Do not require a fixed Sonnet/Opus pass-rate relationship to declare correctness.

16 | Issue-to-environment as an input adapter

Implement later: existing draft `issue_runtime`. Start with **resolved issues linked to a verified fix**. This adds issue-first discovery without duplicating the PR construction engine. An unresolved issue with no known solution needs the `repo_feature` authoring path and additional feasibility work.



Upstream implementation references

[microsoft/SWE-bench-Live curation/swe_task_crawling/](#), MIT: `get_pull_issue_dict.py`, `get_pull_request_content.py`, `get_tasks_pipeline.py` and `build_dataset.py` for issue/PR evidence collection. [R2E-Gym src/r2egym/repo_analysis/add_github_issue_to_commit.py](#) and `collect_issue_content.py` provide commit-linked issue references; Apache-2.0.

Proposed local files: `pipelines/issue_runtime.py`; `synthesis/sources/issues.py`; `reuse curation/sources.py`, `pipelines/pr_runtime.py` and the generalized Tasksmith graph. Update `docs/rfcs/0010-issue-runtime.md` to make resolved/unresolved modes explicit.

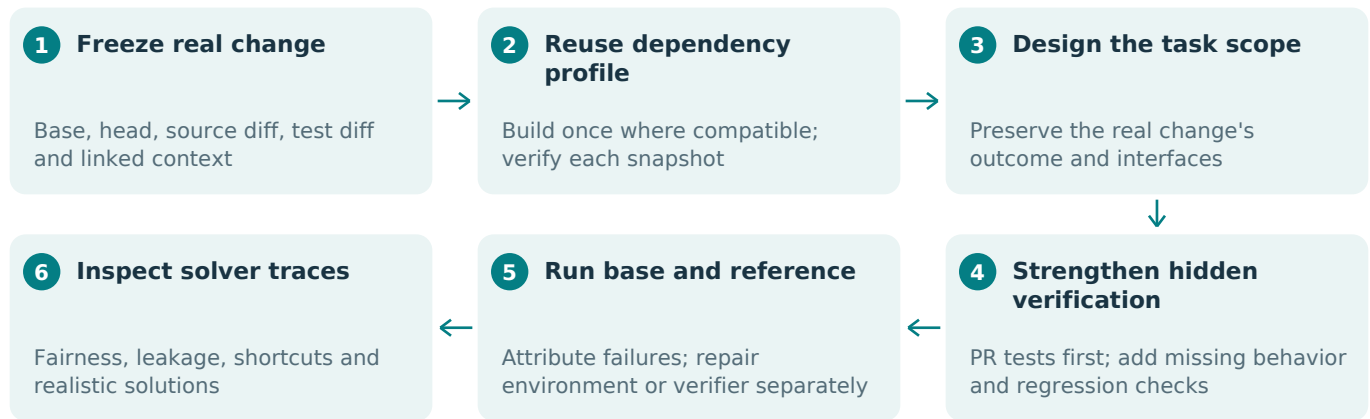
Example task: a client mishandles redirects under a documented option. Give a local reproduction server and the expected observable behavior, while keeping linked fix discussions private.

Milestone: an issue resolves to exactly pinned evidence, its claimed defect is reproducible, and its reference fixes that defect. Duplicate issue/PR inputs map to the same source family so they do not inflate the dataset or leak across splits.

An issue-only input does not justify silently selecting an arbitrary current commit as its initial state. If chronology or linkage is uncertain, preserve the evidence gap and repair source resolution before generation.

17 | Improve the existing PR and commit pipelines

Enhance `pr_runtime` **and** `commit_runtime`; **keep Tasksmith as their deeper construction route.** `pr_diff` can remain a lightweight similarity-oriented product, but similarity reward alone does not meet this project's behavioral acceptance goal.



Upstream implementation map

- [SWE-gen src/swegen/create/task_instruction.py](#): combine PR/issue/test evidence into a task description.
- [SWE-gen src/swegen/create/claude_code_runner.py](#): iterative construction; [src/swegen/tools/validate.py](#): explicit NOP-zero/oracle-one validation.
- [R2E-Gym src/r2egym/repo_analysis/](#): commit collection, execution result analysis and synthetic issue descriptions.
- [SWE-rebench V2 prompts/annotations/](#): interfaces and instruction annotations. These released components do not establish a complete public production miner.

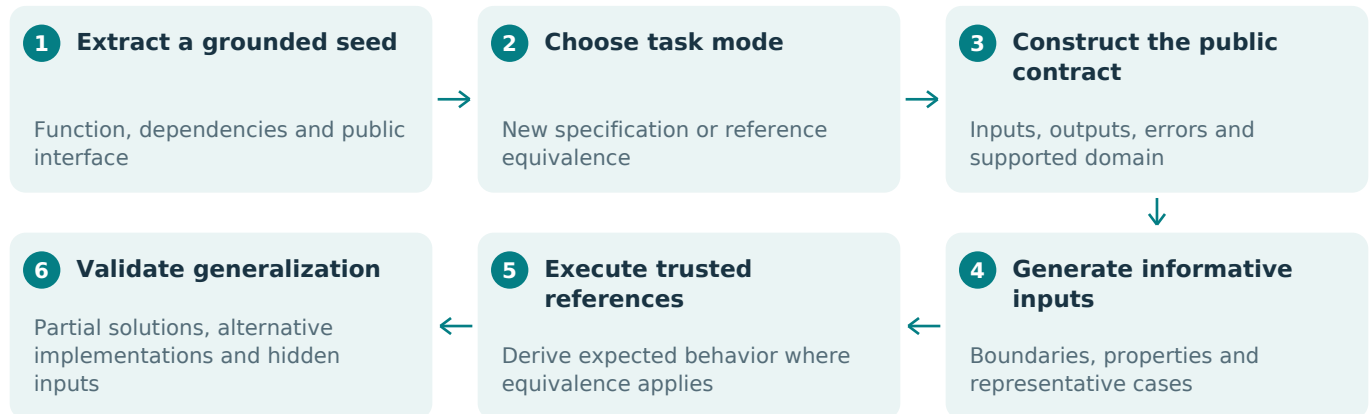
Local files: `pipelines/{pr_runtime,commit_runtime,pr_runtime_validate,_pr_runtime_verifier}.py`; `tasksmith/{build,authoring,specification,validation}.py`; `curation/sources.py`. Unify shared behavior rather than adding top-level pipelines named after every upstream PR project.

Milestone: revisit a fixed PR panel including a missing-test PR and a dependency-changing PR. Each accepted environment must preserve the intended change; one PR cannot be marked converted merely because a related easier task was synthesized. Record unsupported resource requirements as such rather than labeling the PR inherently poor.

PR tests are a starting point. Grade the behavioral contract, allow alternative implementations, and add independent cases when the original tests are narrow. Missing dependency failures and zero collected tests must never be counted as desired baseline regressions.

18 | Improve the existing function pipelines

Enhance `code_instruct` **and** `equivalence_tests`. The first creates a coding task inspired by a snippet; the second asks the learner to reproduce an existing function's observable behavior. Share source extraction and test generation while retaining these different claims.



Upstream implementation references

[r2e-project/r2e](#), MIT: `src/r2e/pat/dependency_slicer/` for context extraction; `src/r2e/generators/testgen/genexec.py` for test generation/execution feedback; `src/r2e/generators/specgen/` for specification generation. Reference execution is appropriate when the original behavior defines the target. It cannot invent the semantics of an unimplemented new feature.

[THUDM/SWE-Dev swedev/testcases/get_descriptions.py](#), `get_testcases.py` and `eval_testcases.py` provide staged behavior-to-test construction. Reuse this as a **verifier-authoring service**, not as a claim that the learner is writing tests.

Local files: extend `pipelines/{code_instruct,equivalence_tests,_function_extractor,_oss_instruct}.py`; add `synthesis/testing/{behavior_cases,reference_execution}.py`. Preserve stated language capabilities; do not silently apply Python AST machinery to other languages.

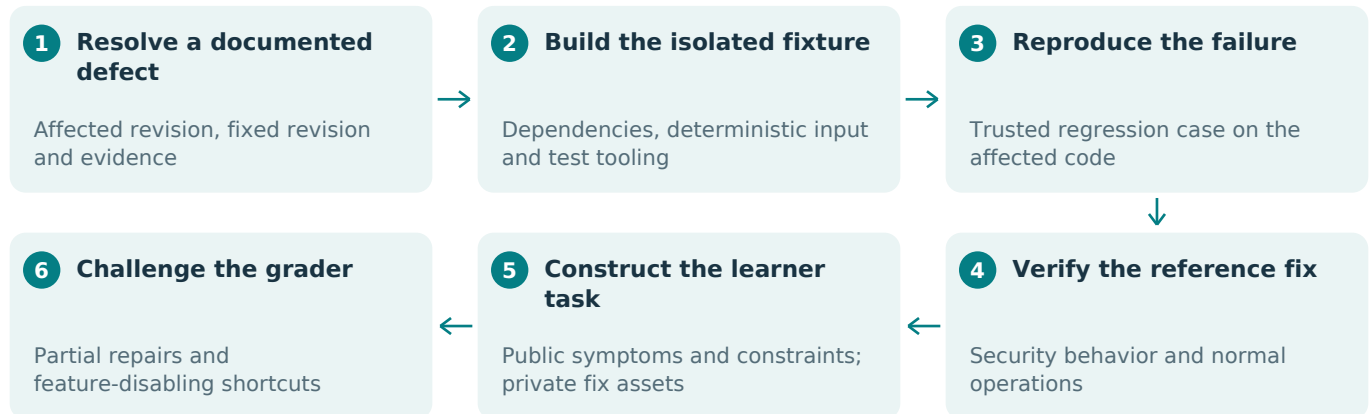
Example task: implement a normalization function with explicit Unicode, whitespace and error behavior. For equivalence mode, sample additional inputs and obtain expected outputs by trusted reference execution. Protect the reference code and output tables from the learner.

Milestone: a valid alternative algorithm passes; a hardcoded response table and a happy-path implementation fail. Generated properties must follow the declared domain rather than accidentally imposing implementation-specific behavior.

An optional performance profile can later add resource constraints to behavior-preserving tasks. It requires a reproducible benchmark protocol and separate correctness gate, described in section 20.

19 | Strengthen security patch environments

Enhance `cve_patches` **with reproducible, isolated regression evidence.** The learner fixes a vulnerability in a local fixture repository. The final reward should require both the intended security property and preservation of ordinary functionality.



Upstream implementation references

[SEC-bench/SEC-bench](#), MIT: `secb/preprocessor/{seed,report,project}.py` for inputs; `build_base_images.py`, `build_instance_images.py` and `templates/` for construction; `secb/evaluator/build_eval_instances.py` for evaluation instances.

[SEC-bench/SecVerifier multi-agent.py](#), MIT, contains builder, reproducer and fixer roles. Its larger OpenHands application is a role/workflow reference; importing the whole stack is unnecessary when Tasksmith already owns orchestration and coding-agent adapters.

Local files: enhance `pipelines/cve_patches.py`; add `synthesis/security/{reproducer, regression, reference}.py`; use shared build profiles and artifact isolation. Run all reproduction against our sandbox fixture.

Example task: restore a parser's bounds handling while preserving valid-input parsing and useful error reporting. Hidden tests include normal inputs, malformed inputs and partial-fix edge cases.

Milestone: the original revision exhibits the intended defect, the reference eliminates it, and a patch that simply disables parsing fails ordinary behavior checks. Harness crashes, sanitizer setup problems and nondeterministic external services are infrastructure issues to resolve before accepting the task.

This is a specialization of an existing pipeline, not a reason to launch another independent CVE mining/build system. Source provenance and redistribution information travel with the task bundle just as they do for ordinary repository tasks.

20 | Optional specialist pipelines and profiles

These additions broaden the product after the core factory works. They should have explicit domain boundaries and their own validation evidence.



Parameterized coding families - proposed `problem_families`. SCALER is a reference for generating executable problem families and difficulty controls. Study [ALEX-nlp/SCALER](#) [SCALER/api_generate_generator_for_environment.py](#), [SCALER/generate_problem_from_environment.py](#) and [recipe/environment/](#). Its reasoning-oriented families are not a ready repository-editing factory. Our adaptation would sample coding instances, bundle a reference and hidden seeds, and emit Harbor tasks. Proposed files: `pipelines/problem_families.py`, `synthesis/families/{generator,instances,oracle}.py`. Validate that unseen seeds have correct expected answers and stay within declared resource bounds.

Performance optimization - a profile, initially under `feature_rebuild` **or** `equivalence_tests`. Preserve semantics while asking for lower latency, memory or another measurable resource. Proposed file: `synthesis/testing/performance.py`. Use fixed hardware, workload families, warmup, repeated measurements and declared tolerances. Reject incorrect outputs before any speed score. This is our design, not a claim that an upstream generator provides a portable benchmark harness. GPU tasks require a tested GPU provider profile.

ML/data workflow tasks - a SETA seed adapter. Start with notebook-derived data processing or evaluation tasks before full training competitions. Proposed file: `synthesis/seeds/adapters/ml_workflow.py`. Freeze datasets, metric definitions and randomness; protect evaluation labels. MLE-Smith is a paper-method reference, with no official generator located in the prior landscape audit. Dataset-dependent grading requires more work than exporting a notebook.

Tool worlds and joint self-play - defer as a separate product surface. AgentWorldModel and SPADE concern generated tool worlds or trained proposer/solver loops. They are relevant research, but importing them does not directly improve a PR-to-coding-task contract. Their training dependencies and output semantics should be assessed in a separate RFC before assigning implementation folders.

21 | The shared verifier and acceptance system

A task is accepted only when its public requirements, actual environment and grader agree. A polished instruction, one successful oracle or a favorable author report alone is insufficient.



Verifier choice follows the behavior: unit tests for pure logic; integration checks for boundaries/services; end-to-end checks for a workflow; property/metamorphic checks for broad input spaces; controlled measurements for performance. A single test file can contain strong coverage, and hundreds of assertions can still miss the real requirement.

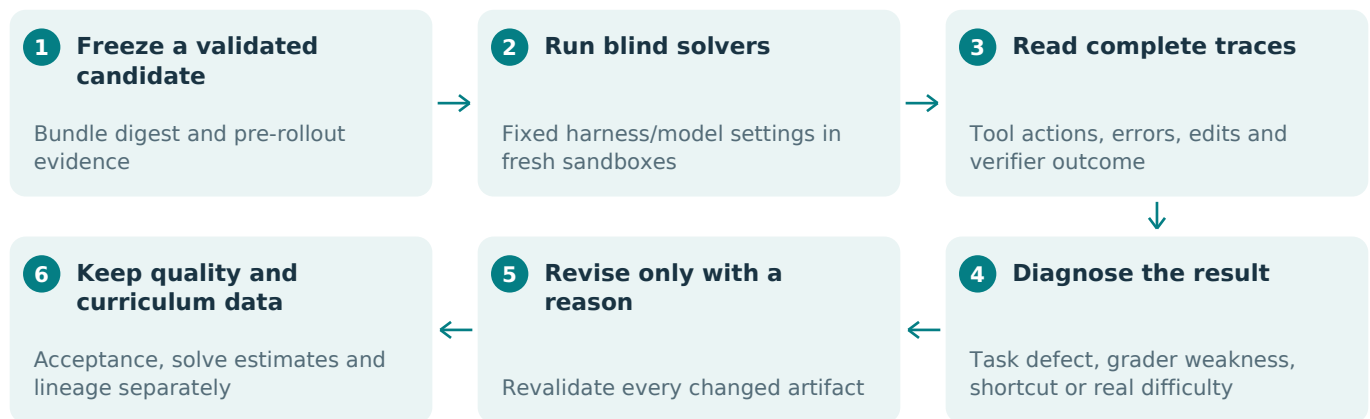
Reusable upstream code: [SWE-Dev swedev/testcases/](#) for missing-test construction; [SWE-Mutation framework/mutation.py](#) for wrong-solution challenges; [harden-v0 harden/loop.py](#), [agent.py](#), [workspace.py](#) and [precheck_cache.py](#) for hacker/fixer/legitimate-solver loops and replay. These are shared quality components, not independent source-generation pipelines.

Local implementation: extend `tasksmith/validation.py`; add `tasksmith/{verifier_authoring,challenge,quality_report}.py`; version a rubric under `tasksmith/rubrics/`. Harbor's installed checker models expose `pass/fail/not_applicable` with explanations. Treat static LLM review as evidence about clarity and alignment, not a replacement for execution.

Proposed report: deterministic gates plus an anchored 0-4 review vector for clarity, coverage, realism, independence and robustness, with evidence and reasons. Do not average away a failed correctness gate. Default RL rewards remain deterministic; an explicit semantic-judging mode must state where and why a model affects reward.

Baseline failure means the new requirement is unsatisfied. Existing regression tests may and should pass. Never inherit a rule that every no-op test must fail. A partial solution must miss at least one essential requirement; an alternative correct implementation should pass regardless of patch similarity.

22 | Rollout calibration and honest yield



Use Sonnet and Opus as a configured evaluation panel, recording exact model IDs when the experiment runs. Keep Pi/OpenCode author-harness comparisons separate from solver-model comparisons. Begin with paired runs on the same tasks; add repetitions where nondeterminism or uncertainty could change a decision.

Failure does not automatically invalidate a task. A strong solver failing because it overlooks a documented edge case is difficulty evidence. Failing because the task omitted a required convention is an instruction defect. Passing by changing a status file while leaving the service broken exposes a grader weakness. The trace determines the repair target.

TaskPilot motivates learner-feedback revision; CalibForge motivates inspecting cross-solver behavior; Environment Evolution motivates lineage-aware task selection. Their methods are useful references, but no complete public generator for these three was located. [TaskPilot](#), [CalibForge](#), [Environment Evolution](#).

Local files: generalize `tasksmith/{trials,review,evidence_projection,finalize}.py`; add `tasksmith/calibration.py`. Store trial model/harness versions, prompt hashes, token usage, provider cost and failure categories. Reuse construction evidence for an unchanged revision, but do not reuse a previous learner filesystem as a fresh rollout.

Required campaign measurements

- Faithful PR conversion: accepted unique PR tasks divided by the frozen eligible PR panel; retain an all-input view too.
- Synthetic generation: accepted unique tasks per seed and attempt; report duplicates and failed revisions.
- Stage yield: proposals, built environments, valid references, robust verifiers and accepted tasks as separate counts.
- Cost: bootstrap, ideation, authoring, test repair, static review, challenges, rollouts and idle resources separately; include failed attempts.
- Utility: model solve rates with sample sizes, task diversity and later learning gains if training is actually measured.

The first POC demonstrates a working construction contract. It cannot establish universal one-environment-per-PR yield or training improvement. Preserve that distinction while optimizing repair paths instead of weakening acceptance.

23 | Where the implementation belongs

Use the existing package layout. Add method-specific code under a proposed `synthesis/` package, thin registered pipeline entry points, and shared Tasksmith services. Paths marked **new** do not exist yet.

Area	Existing location	Proposed change
Input and options	<code>src/repo2rlev/spec/{input,options}.py</code>	Add discriminated source inputs, recipe options and backwards-compatible normalization
Pipeline dispatch	<code>src/repo2rlev/pipelines/{base,__init__}.py, cli.py</code>	Register families; remove repository-only preflight for non-repo inputs
Generation methods	Existing pipeline modules	New: <code>src/repo2rlev/synthesis/</code> with terminal, seeds, evolution, bugs, features, environment, traces, testing and sources
Durable execution	<code>src/repo2rlev/tasksmith/{pipeline,models,state}.py</code>	Generalize source/operation identity; route to recipe subgraphs
Remote runtime	<code>tasksmith/{providers,conformance,harbor_environments}.py</code>	Capability profiles, remote builds and verified lifecycle
Dependency reuse	<code>src/repo2rlev/bootstrap/</code>	Generalize profiles/cache; route standard CLI through remote backend
Coding agents	<code>curation/external_agent.py, curation/runtimes/</code>	Reuse Pi/OpenCode; isolate stage tools and record versions
Output and publishing	<code>emitter/harbor.py, tasksmith/emit.py, hub.py</code>	General task bundle, binary assets, trusted grading and provenance
Validation	<code>tasksmith/{validation,review,trials}.py</code>	Add verifier authoring, challenge suites and calibration
Tests and docs	<code>tests/, docs/rfcs/, docs/pipelines/</code>	Per-family contracts, provider checks and documented acceptance evidence

Proposed method layout: `synthesis/terminal/{tmax,endless}.py`; `seeds/seta.py`; `evolution/seta.py`; `bugs/swesmith.py`; `features/{trace_graph,proposal}.py`; `environment/{bare,cli_gym}.py`; `traces/terminalworld.py`; `testing/{variants,grader,reference_execution}.py`.

Reference checkouts should be pinned under `references/<project>/` for development. Some already exist, including SWE-smith, RepoLaunch, R2E-Gym, r2e and Harbor; do not overwrite them blindly. Record upstream commit, selected files and license in a proposed `references/upstreams.toml`. Import or vendor only the needed code with attribution; avoid shipping training stacks and arbitrary upstream upload defaults.

The cookbook `docs/contributing/ADDING_A_PIPELINES.md` requires an RFC for each new registered name, plus enum/options/registry additions and a pipeline contract test. Update existing drafts rather than creating competing definitions.

24 | A staged implementation and validation plan

Ship complete vertical slices. The number of adapters matters less than proving that the same controller can construct, verify and replay different task types without a separate workaround for each one.

Stage	Implementation goal	Evidence required to advance
1. Contracts	Generic sources and bundles; compatible legacy input	Legacy PR, script task and binary fixture round-trip; full-bundle hashing; pinned Harbor parsing
2. Remote runtime	Both Modal and Daytona; dependency reuse	Fresh build/start/test/teardown on both; compatible cache hit; invalidation on dependency change
3. Common quality	Reference/baseline, collection checks, artifact isolation and challenges	Known valid task admitted; leaked, uncollectable and shortcut-rewarding fixtures rejected for the right reasons
4. Terminal comparison	TMax and Endless recipes under one family	Same five seeds, same resource/solver policy, complete attempt ledger and accepted task bundles
5. SETA routes	Seed2Synth and immutable evolution	Notebook/Q&A sources work offline; independently accepted children preserve parent hashes
6. Repository methods	Bug injection and feature reconstruction	Healthy/broken/restored evidence; meaningful scope and hidden partial-fix checks
7. Specialized tasks	Environment repair, trace reconstruction and learner-written tests	Recipe-specific milestones from sections 12-14; no bypass of shared gates
8. Expansion	Novel features, issues and optional specialist profiles	New contracts validated; diversity and cost measured on an expanded fixed panel

The five-seed terminal comparison is a practical first experiment, not a replacement for the separate five-PR panel. Keep both panels if the objective includes both faithful PR conversion and free-form synthesis. Only claim a provider/recipe combination works after it has run; abstract API compatibility is insufficient.

Prompt deliverables at each authoring stage

Discovery returns evidence-backed behavior and resource needs. Ideation returns several task/verifier designs with tradeoffs. Specification returns observable requirements and an assertion map. Construction returns files and a reproducible recipe. Verification repair returns a diagnosed mismatch and targeted change. Trace review returns cited observations and a failure classification. None of these roles can set the final acceptance flag directly.

Definition of a completed pipeline

Registered input/options, documented recipe provenance, remote execution, resumable evidence, a representative accepted task set, deterministic grading, blind solver traces, failure reporting and a replayable Harbor export. A wrapper that writes folders or asks the author whether its task passed is incomplete.

25 | Upstream source register

These are source-inspected revisions, not runtime-certified integrations. Links in earlier sections point to immutable versions. License labels refer to the inspected code repositories; dataset and fixture redistribution terms require their own provenance records.

Upstream	Revision prefix / code license	Intended integration
hamishivi/tmax	7387d2f9 / Apache-2.0	Terminal compositional sampler, fixtures and exporter
kanishkg/endless-terminals	99f4c74b / Apache-2.0	Staged terminal synthesis baseline
camel-ai/seta	e4715b01 / Apache-2.0	Seed adapters and evolution prompts
SWE-bench/SWE-smith	9b74ac08 / MIT	Repository mutations, profiles and Modal route
Hambaobao/SWE-Flow	7da5b046 / MIT	Test-guided feature reconstruction
Hambaobao/SWE-Flow-Trace	e7251448 / MIT	Runtime dependency tracing
LiberCoders/CLI-Gym	48bb920b / MIT	Environment corruption and repair construction
EuniAI/TerminalWorld	784698ba / Apache-2.0	Recording-derived tasks and refinement
abundant-ai/SWE-gen	14e185f4 / Apache-2.0	PR instructions, construction and paired validation
microsoft/RepoLaunch	db3eb23a / MIT	Setup graph, test parsers and image reuse
TIGER-AI-Lab/SWE-Next	b55c0841 / Apache-2.0	Dependency profiles and commit execution
SWE-rebench/SWE-rebench-V2	c71902a8 / MIT; partial release	Installer/parser/annotation prompts
R2E-Gym/R2E-Gym	0d94c4eb / Apache-2.0	Commit mining and execution analysis
microsoft/SWE-bench-Live	9a273490 / MIT	Issue and PR evidence collection
THUDM/SWE-Dev	72917b61 / MIT	Missing-test generation and revision
r2e-project/r2e	bcbcd156 / MIT	Dependency slicing and reference execution
Sunny4Coding/SWE-Mutation	c9485dc1 / MIT	Wrong-solution/verifier challenges
few-sh/harden-v0	342b8474 / Apache-2.0	Adversarial repair and replay
SEC-bench/SEC-bench	31eb4348 / MIT	Reproducible security task construction
SEC-bench/SecVerifier	93abf590 / MIT	Builder/reproducer/fixer role workflow
ALEX-nlp/SCALER	60c6c503 / Apache-2.0	Parameterized environment generation
DataArcTech/DataArc-SynData-Toolkit	2a1d65ec / Apache-2.0	Partial few/self/evol Harbor implementation

Local environment inspected: Harbor 0.20.0, Daytona 0.198.0, Modal 1.5.5 and LangGraph 1.2.11. Current online documentation can describe newer contracts: for example, the installed Harbor default schema is 1.3 while the live task example shows 1.4. Bind conformance tests to selected dependency versions before adopting new fields.

26 | Paper-method recipes and remaining boundaries

Some methods should inform our roadmap without being described as existing open-source adapters. The following availability statements reflect the official repositories and source releases located in the September 10 landscape/source audit; they are not proof that code can never appear elsewhere.

Method	Useful addition	Implementation status and destination
TaskPilot	Repository task proposals revised from learner traces	No official generator located; proposed repo_feature and shared calibration
Environment Evolution	Scenario/skill/length transformations and lineage selection	No official generator located; future synthesis/evolution/ operators and scheduler
Recursive Task Synthesis	Extend solutions, then realign environment/tests/instruction	Prompts/data available; generator not located; future evolution recipe
CalibForge	Cross-solver trace diagnosis and task repair	Task release and minimal project repository; no generator located; shared calibration
CLI-Universe	Separate first-draft solution and test authors	No official generator located; shared role-isolation design
SkillSynth / Meta-Task	Skill-grounded planning and sandboxed task authoring	No official generators located; terminal/seed prompt and graph references
SSR / Socratic-SWE	Learned proposer/solver or gradient-based selection	Research requiring learner training access; not equivalent to alternating API prompts
SERA	Synthesize useful coding trajectories	Public trajectory-generation code; ancillary data output, not proof of behavioral task validity

Primary method references: [TaskPilot](#), [Environment Evolution](#), [Recursive Synthesis](#), [CalibForge](#), [CLI-Universe](#), [SkillSynth](#), [Meta-Task](#), [SSR](#), [Socratic-SWE](#), [SERA source](#).

Network and artifact boundary: build-time dependency access and learner-time access are separate. Stage allowed assets before the episode where possible. A host allowlist for Hugging Face does not restrict access to one bucket path; use scoped artifact delivery or a controlled fetch proxy if live fetching is essential. Scrub git history, solution files and answer-bearing author logs from the learner state; grade against trusted artifacts. Harbor's host/network capabilities depend on the provider and pinned version. [Harbor task network contract](#).

Research boundary: source trees, consequential code paths and local contracts were inspected; upstream generators and cloud profiles were not executed. Counts, costs and yield forecasts have deliberately not been inferred from code availability. The next empirical milestone is the generic contract plus two-provider smoke, followed by a controlled TMax/Endless comparison and SETA adapters.